DOCUMENTO TÉCNICO · V1.0

WorkUp
Esquema de base de datos explicado

Qué es cada tabla, por qué existe y para qué se usa cada propiedad.
Derivado de `001_schema.sql`, `002_client_account_pending_names.sql` y `003_outbox_dispatch.sql`, con el vocabulario de negocio del SRS v3.0.

Proyecto: WorkUp — sistema de gestión para emprendimientos que prestan servicios por turno
Fuente de verdad del modelo: `WorkUp/WorkUp.Data/Sql/001_schema.sql`
Complementos: `002_...` (cuentas de cliente) y `003_...` (despacho de avisos)
Motor: SQL Server · Claves `INT IDENTITY` · Enumeraciones `TINYINT` · Fechas `DATETIME2(0)` · Dinero `DECIMAL(12,2)`

1. Cómo leer este documento

El esquema no es una traducción literal del relevamiento: es la traducción del **ciclo de negocio** de un salón de turnos. Cada tabla responde a una pregunta concreta que la dueña se hace todos los días:

Pregunta del negocio	Dónde se responde
¿Quién es mi negocio y cómo se opera?	Businesses, OwnerAccounts, SystemSettings
¿Qué vendo, a qué precio, con qué duración y cupo?	Services, MembershipPrices
¿Quién atiende y cuándo está disponible?	Workers, WorkerSchedules, WorkerBreaks
¿Qué se repite todas las semanas?	GridClasses
¿Qué pasa en un día concreto?	Appointments, Bookings
¿Quiénes son mis clientes y cómo entran solos?	Clients, ClientAccounts, AccountTokens, AccountInvitations, AppointmentLinks
¿Quiénes abonan y cuánto cubren?	Enrollments, EnrollmentFixedClasses, MembershipPeriods
¿Cuánta plata entró, se anuló o se devolvió?	Payments, PaymentAnnulments, Refunds, Expenses
¿Qué tengo en el depósito?	Supplies

Pregunta del negocio	Dónde se responde
¿Qué avisos salieron y cuáles quedaron pendientes?	Notifications

CONVENCIONES DEL ESQUEMA

- **Nombres en inglés, PascalCase.** El dominio se modela en inglés; la interfaz habla en español rioplatense.
- **Claves `INT IDENTITY`.** Enteros autoincrementales: baratos de indexar y suficientes para el volumen de un local.
- **Enumeraciones como `TINYINT`.** Los estados se guardan como número, con el significado fijado en `WorkUp.Domain.Enums`. **Nunca se renumera** un miembro existente: renumerar reescribiría la historia.
- **`BusinessId` en toda tabla de inquilino.** Un negocio = un inquilino. Es la columna que hace que todas las consultas puedan filtrarse por negocio, y la base de la separación de datos entre emprendimientos.
- **Fechas de negocio en `DATETIME2(0)`.** Se guarda la hora local del local (Argentina), con precisión de segundos, sin zona horaria: el horario de atención es una propiedad del negocio, no del servidor.
- **El dinero es `DECIMAL(12,2)`.** Nunca `float`: los importes se suman en caja y un redondeo binario produce diferencias de centavos que rompen el arqueo.
- **Los estados se derivan de los hechos.** Los códigos usan la columna y —en un cuadro chico— la nota `D-xx`/`RF-xx`/`Regla n` que la justifica en el SRS v3.0.

2. Núcleo del negocio

Businesses — el negocio como inquilino

Por qué existe: WorkUp es multi-rubro y multi-cliente (D-04). El alcance se acota a servicios prestados por turno con trabajador asignado; lo que cambia entre una peluquería y un estudio de yoga son los **valores**, no las **reglas**. Por eso existe una sola tabla de negocio que guarda tanto la identidad como los parámetros de operación que consume el resto de los requerimientos `RF-01`.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador del inquilino. Es el valor que viaja en `BusinessId` por todo el esquema.
`Name`	`NVARCHAR(120)`	Nombre comercial del local, tal como lo lee la dueña y lo ve el cliente en la superficie pública de reserva.
`Slug`	`VARCHAR(60) UNIQUE`	Identificador legible para la URL pública `/r/{slug}`. Es único globalmente porque las rutas públicas no llevan sesión: el slug es lo que permite resolver el negocio antes de autenticar.
`ContactPhone`	`VARCHAR(30)`	Teléfono de contacto del negocio. Se muestra al cliente y da contexto cuando algo falla; no es el teléfono con el que se identifica a un cliente (ese vive en Clients).
`NotificationEmail`	`NVARCHAR(254) NULL`	Casilla a la que se dirigen el resumen diario de agenda y el aviso de revisión de existencias `RF-17`. Es nulable porque un negocio puede operar sin correo configurado y seguir usando el panel.
`AllowsWalkIns`	`BIT NOT NULL, 1`	Habilita la atención sin turno `Regla 15`. Por defecto `1` porque el caso relevado (peluquería) la usa; un negocio de grilla puede apagarla.
`ReminderLeadHours`	`INT NOT NULL, 24`	Anticipación con la que se emite el recordatorio de turno `RF-17`. Vive en el negocio, no en cada turno: es una política de aviso, y el `CHECK (1..168)` impide valores absurdos (0 o más de una semana).
`DailySummaryTime`	`TIME(0) NOT NULL, 21:00`	Hora de emisión del resumen de agenda del día. 21:00 es el cierre razonable de una jornada de local: la dueña ya terminó y todavía está despierta.
`StockReviewEveryDays`	`INT NOT NULL, 7`	Periodicidad del aviso de revisión de existencias `RF-18`. El sistema **avisa** que hay que mirar el depósito, no calcula consumo (D-03, Regla 8): el número declara cada cuánto insistir.
`CreatedAt`	`DATETIME2(0)`	Fecha de alta del negocio. Trazabilidad y corte natural para reportes históricos.

OwnerAccounts — la cuenta única de acceso al panel

Por qué existe: hay **una sola cuenta por negocio** (D-06, Restricción 5). Trabajadoras y profesoras son **datos**, no usuarios: cobran y entregan lista, pero registra la dueña. Sin esta restricción, el esquema tendría roles y permisos que el relevamiento no pide.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador de la cuenta. Se guarda en la sesión del panel.
`BusinessId`	`INT FK → Businesses`	Vínculo con el negocio. Junto con `UQ_OwnerAccounts_Business` garantiza en la base lo que la aplicación ya impone: una cuenta y solo una por negocio.
`Username`	`NVARCHAR(60) UNIQUE`	Nombre de usuario de login. Único global porque el panel se autentica sin elegir negocio primero: el usuario resuelve la cuenta y de ahí el negocio.
`PasswordHash`	`NVARCHAR(400)`	Hash de la contraseña (nunca la contraseña). 400 caracteres cubren con holgura los formatos modernos con salt y parámetros de costo.
`CreatedAt`	`DATETIME2(0)`	Alta de la cuenta. Trazabilidad mínima de acceso.

Restricción deliberada: **no hay tabla de roles**. La ausencia es la decisión.

SystemSettings — pares clave/valor de la instalación

Por qué existe: reserva un lugar para ajustes que son de **toda la instalación** y no de un negocio, sin obligar a un `ALTER TABLE` por cada uno. El caso previsto fue el desfasaje del reloj simulado (hipótesis 11 del plan de demo).

Propiedad	Tipo	Uso
`Key`	`VARCHAR(60) PK`	Nombre del ajuste. Es la clave primaria: no hay dos valores para el mismo nombre.
`Value`	`NVARCHAR(400)`	Valor como texto. Guardar todo como texto mantiene la tabla genérica; la conversión al tipo real la hace el código que lo lee.

Estado real: hoy el reloj simulado mantiene su desfasaje en memoria y no escribe aquí (así lo documenta `SimulatedBusinessClock`). La tabla existe como punto de extensión y queda vacía. Un reinicio del proceso es el reset seguro. Conviene leerla como `RESERVADA` antes que como funcionalidad en uso.

3. Catálogo y personal

Services — el tarifario

Por qué existe: el precio, la duración y el cupo máximo se definen **a nivel del negocio** `Regla 6`. De este registro salen las dos formas de atención: un turno individual es el caso particular de un servicio con *cupo máximo 1*, y una clase grupal es el mismo servicio con cupo mayor.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador del servicio. Lo referencian Appointments y GridClasses.
`BusinessId`	`INT FK → Businesses`	Aísla el tarifario por negocio: dos locales pueden tener un "Corte" con precios distintos.
`Name`	`NVARCHAR(100)`	Nombre visible del servicio (por ejemplo, "Corte y peinado", "Yoga integral"). No se fuerza único: dos servicios distintos pueden llamarse igual y distinguirse por duración o precio.
`Price`	`DECIMAL(12,2)`	Precio de lista vigente. Es el **valor por defecto**: cuando una anotación o un pago se crean, este precio se copia a `AgreedPrice`. Cambiar el tarifario después nunca reescribe lo ya cobrado. `CHECK (Price > 0)` impide servicios gratuitos, que además no existen en el relevamiento.
`DurationMinutes`	`INT`	Duración del servicio. Es lo que usa el cálculo de disponibilidad para saber cuánto ocupa el turno `Regla 1`. `CHECK (> 0)` porque una duración nula haría infinitos los huecos libres.
`MaxCapacity`	`INT`	Cupo máximo simultáneo del servicio `Regla 2`. Con 1 el sistema crea un turno por reserva; con más de 1, los turnos se generan desde la grilla y los clientes se anotan encima. `CHECK (>= 1)`: el cupo 0 no tiene sentido operativo.
`IsActive`	`BIT NOT NULL, 1`	Baja lógica del tarifario. Un servicio con historia (turnos y cobros) **no se borra nunca**: se desactiva para que deje de ofrecerse sin romper los turnos que ya lo referencian.

MembershipPrices — el precio del abono por frecuencia

Por qué existe: el abono es mensual y reserva un lugar en las clases fijas `D-13, Regla 16`. Su precio depende de **cuántas veces por semana** asiste el alumno, no del servicio. Por eso es una tabla aparte y no una columna de Services.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador de la fila de tarifa.
`BusinessId`	`INT FK → Businesses`	Tarifario de abono propio de cada negocio.
`WeeklyFrequency`	`INT`	Clases fijas por semana a las que da derecho (1 a 7). Junto con `BusinessId` es único: no puede haber dos precios para la misma frecuencia. El rango 1..7 se corresponde con los siete días de la semana.
`Price`	`DECIMAL(12,2)`	Precio mensual del abono de esa frecuencia. Es el valor que se copia a `Enrollments.AgreedPrice` al inscribir.

Workers — quién atiende

Por qué existe: la disponibilidad se calcula **por trabajador** `Regla 1` y el cliente elige con quién atenderse antes de ver horarios `D-01`. Los trabajadores son datos operativos, no usuarios (D-06).

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador del trabajador. Lo referencian horarios, pausas, grilla y turnos.
`BusinessId`	`INT FK → Businesses`	Aísla la plantilla por negocio.
`FirstName / LastName`	`NVARCHAR(60) c/u`	Nombre y apellido separados: la agenda y las listas de clase necesitan el nombre corto, y el orden alfabético y los reportes el apellido.
`Phone`	`VARCHAR(30)`	Contacto del trabajador. Es dato administrativo del local; no participa del cálculo de disponibilidad.
`Cuil`	`VARCHAR(13)`	Identificación laboral. Es único por negocio: es el dato que distingue a dos personas con el mismo nombre y apellido, típico en planteles chicos.
`IsActive`	`BIT NOT NULL, 1`	Baja lógica. Igual que en Services: un trabajador con turnos y cobros históricos se desactiva, no se borra. Al desactivarlo deja de ofrecerse para reservar sin destruir el historial de la agenda.

WorkerSchedules — el horario de atención por día de la semana

Por qué existe: es la base del cálculo de disponibilidad. El horario de atención de cada trabajador se repite semanalmente, por eso el modelo guarda una fila por día en lugar de un calendario de fechas.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador del tramo de horario.
`WorkerId`	`INT FK → Workers ON DELETE CASCADE`	Dueño del horario. El `CASCADE` es seguro porque un horario no tiene historia propia: si se borra el trabajador (situación de corrección, no de baja), los tramos de disponibilidad no tienen sentido sueltos.
`DayOfWeek`	`TINYINT 0..6`	Día de la semana, siguiendo `System.DayOfWeek`: 0 = domingo … 6 = sábado. Se alinea con la enumeración de .NET para no traducir en cada consulta.
`StartTime / EndTime`	`TIME(0) c/u`	Extremos del tramo de atención. `CHECK (EndTime > StartTime)` rechaza tramos vacíos o invertidos, que producirían huecos negativos en el cálculo de disponibilidad.

Varias filas por día son válidas: permiten representar turnos partidos (mañana y tarde) sin ningún modelo adicional.

WorkerBreaks — las pausas no laborables

Por qué existe: el horario libre se calcula descontando **pausas no laborables** `Regla 1`. Sin esta tabla, el almuerzo de una trabajadora aparecería como tiempo reservable y el cliente podría sacar un turno en ese hueco.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador de la pausa.
`WorkerId`	`INT FK → Workers ON DELETE CASCADE`	Trabajador al que pertenece la pausa. Mismo criterio de borrado que los horarios: es dato de configuración, no historial.
`DayOfWeek`	`TINYINT 0..6`	Día de la semana de la pausa. Recurrente, igual que el horario: "los martes de 13 a 14" se declara una vez.
`StartTime / EndTime`	`TIME(0) c/u`	Ventana de la pausa. El `CHECK (EndTime > StartTime)` impide una pausa de duración nula o negativa.

ExpenseTypes — el catálogo de tipos de gasto

Por qué existe: `RF-01` pide registrar "los tipos de gasto que usa el negocio" al dar de alta el negocio, y `RF-09` clasifica cada gasto con uno. Es un catálogo chico y propio de cada negocio, no una enumeración fija del sistema: la clasificación de gastos es del rubro.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador del tipo de gasto. Lo usa Expenses.
`BusinessId`	`INT FK → Businesses`	Catálogo propio de cada negocio.
`Name`	`NVARCHAR(80)`	Nombre del tipo ("Productos", "Alquiler", "Servicios"). Único por negocio: dos tipos con el mismo nombre harían ambiguo el reporte de resultados.
`IsActive`	`BIT NOT NULL, 1`	Baja lógica. Un tipo con gastos históricos se desactiva para que no se siga eligiendo, pero se conserva para que los gastos viejos sigan clasificados.

4. Grilla semanal y agenda

GridClasses — la grilla recurrente

Por qué existe: la grilla **se repite todas las semanas** `D-05`. Esta tabla guarda la regla ("los martes a las 19 hay yoga con Lucía"), no las clases concretas. De aquí el sistema **genera** las sesiones de cada semana: guardar la recurrencia una sola vez es lo que permite que suspender una clase no altere la grilla `Regla 9`.

Propiedad	Tipo	Uso
Id	INT IDENTITY PK	Identificador de la clase de grilla. Lo referencian Appointments (sesión generada) y EnrollmentFixedClasses (lugar fijo del alumno).
BusinessId	INT FK → Businesses	Aísla la grilla por negocio.
ServiceId	INT FK → Services	Qué se dicta. De aquí salen la duración y el cupo máximo de cada sesión generada.
WorkerId	INT FK → Workers	Quién dicta la clase. La profesora queda determinada por la clase elegida `D-01, R12`.
DayOfWeek	TINYINT 0..6	Día de la semana de la clase recurrente. Misma convención que los horarios (0 = domingo).
StartTime	TIME(0)	Hora de inicio semanal. El fin de la sesión no se guarda aquí: se deriva de la duración del servicio, así un cambio de duración no obliga a reescribir la grilla.
IsActive	BIT NOT NULL, 1	Si la regla sigue vigente. Desactivarla deja de generar sesiones nuevas sin borrar las semanas ya generadas.

Appointments — el turno

Por qué existe: es la pieza central del modelo y la fuente de la conclusión del relevamiento: **un turno individual es un servicio con cupo 1**, y una clase de la semana es el mismo turno con cupo mayor. La tabla guarda **el hecho concreto** ("el martes 12 a las 19 hay una clase de yoga") y su estado global; el estado de cada cliente vive aparte, en Bookings `Regla 3`. También sostiene la decisión **D-09**: la atención sin turno *es* un turno, creado al atender con origen `Sin reserva`, porque todo lo posterior es idéntico y hoy quedan cobros sin turno que explicar `Regla 12`.

Propiedad	Tipo	Uso
Id	INT IDENTITY PK	Identificador del turno. Lo referencian Bookings, ClientNotes y Payments.
BusinessId	INT FK → Businesses	Aísla la agenda por negocio; es la columna del índice de día.
Kind	TINYINT — AppointmentKind	1 = *Individual* (turno o atención sin turno), 2 = *ClassSession* (clase de la semana generada desde la grilla). Es la columna que decide qué forma tiene el registro y qué `CHECK` aplica.
ServiceId	INT FK → Services	Qué se presta. Determina duración, cupo de origen y precio de lista.
WorkerId	INT FK → Workers	Quién atiende. Es la dimensión del índice `IX_Appointments_WorkerDay`: la validación de superposición y el cálculo de disponibilidad se hacen siempre por trabajador `Regla 1`.
GridClassId	INT NULL FK → GridClasses	Regla de la que nació la sesión. Nulo en turnos individuales. Es la clave de la **generación idempotente** de la semana: `UX_Appointments_GridSession` impide crear dos veces la misma sesión `RF-06`.
StartsAt / EndsAt	DATETIME2(0) c/u	Inicio y fin concretos del turno, en hora local del local. Se desnormaliza el fin (en lugar de derivarlo) porque la duración realmente prestada puede diferir del tarifario, y porque el cálculo de superposición debe ser una comparación de columnas indexadas, no un cálculo de expresiones. `CHECK (EndsAt > StartsAt)`.
MaxCapacity	INT	**Copia** del cupo del servicio al momento de crear el turno. Es una copia deliberada: cambiar el tarifario mañana no debe alterar cuántos lugares tenía esta clase. `CHECK (>= 1)`.
Origin	TINYINT NULL — AppointmentOrigin	1 = *Reservado*, 2 = *Sin reserva*. Nulo en clases. Es el campo que registra el walk-in como turno de primera clase `D-09, Regla 12`: el cobro de atención siempre tiene un turno que lo explica.
Status	TINYINT, 1 — AppointmentStatus	1 = *Programado*, 2 = *Suspendido*. Es el estado **del turno** (la clase entera), no del cliente: cada alumno conserva su propia asistencia en Bookings.
SuspensionReason	NVARCHAR(300) NULL	Motivo de la suspensión de una clase (feriado, sala ocupada). Explica ante el alumno por qué la clase no se

Propiedad	Tipo	Uso
		dictó, sin cambiar la grilla `Regla 9`.
SuspendedAt	DATETIME2(0) NULL	Cuándo se suspendió. Distingue la suspensión decidida antes de la clase de la registrada después, y da trazabilidad a quién avisar.
CreatedAt	DATETIME2(0)	Cuándo se creó el turno. Es la referencia para ordenar altas del día y para auditar la generación semanal.

TRES RESTRICCIONES QUE EXPRESAN REGLAS DE NEGOCIO

- `CK_Appointments_KindShape`: un turno individual tiene origen y cupo exactamente 1, y no viene de grilla; una clase no tiene origen. **Por qué:** hace imposible en la base un registro con forma contradictoria ("un turno individual derivado de la grilla", "una clase reservada"), que la aplicación tendría que rechazar una y otra vez.
- `CK_Appointments_Suspension`: solo una clase puede suspenderse `RF-10`. **Por qué:** un turno individual no se "suspende": se cancela la anotación, que es un hecho distinto y vive en Bookings.
- `UX_Appointments_GridSession` (único filtrado sobre `GridClassId, StartsAt`): una sesión por clase de grilla y hora. **Por qué:** es lo que permite regenerar una semana sin duplicar clases `RF-06`. El filtro parcial deja libres los turnos individuales, que sí pueden repetir hora.

Índices: `IX_Appointments_WorkerDay` cubre el caso de uso más caliente —"la agenda de una trabajadora en un día"— incluyendo `EndsAt` y `Status` para resolver superposiciones sin ir a la tabla. `IX_Appointments_BusinessDay` cubre el resumen diario y los reportes por negocio.

5. Clientes y acceso del cliente

Clients — la ficha del cliente

Por qué existe: es la ficha que reemplaza la hoja suelta de la libreta. El cliente se identifica por su **email** dentro de un negocio `Regla 7` y `D-02`: reservar no requiere cuenta, y el teléfono es la vía de contacto rápida.

Propiedad	Tipo	Uso
Id	INT IDENTITY PK	Identificador del cliente. Lo referencian anotaciones, notas, inscripciones y pagos.
BusinessId	INT FK → Businesses	La ficha pertenece a un negocio. La misma persona que va a dos locales distintos son dos fichas distintas: cada local es su propia libreta.
FirstName / LastName	NVARCHAR(60) c/u	Nombre y apellido separados, por la misma razón que en Workers: listas cortas y orden alfabético.
Email	NVARCHAR(254)	Identidad del cliente dentro del negocio `Regla 7`. Se guarda en minúsculas y `UQ_Clients_Email` lo hace único por negocio: es lo que permite reconocer al cliente que vuelve sin crear duplicados. 254 es el máximo de una dirección de correo válida.
Phone	VARCHAR(30) NULL	Vía de contacto del día a día ("te aviso cuando se desocupa"). Nulable porque el cliente puede reservar solo con email `D-02`.
HealthNotes	NVARCHAR(1000) NULL	Observaciones permanentes de salud o cuidado (alergias, lesiones). Es un campo de la ficha, no de la anotación, porque el dato trasciende al turno puntual.
CreatedAt	DATETIME2(0)	Alta de la ficha. Sirve para distinguir clientes nuevos de recurrentes en los reportes y para ordenar el alta.

ClientAccounts — la cuenta opcional de autoservicio

Por qué existe: la reserva no requiere cuenta (D-02), pero una cuenta opcional habilita el autoservicio `RF-25`: ver turnos, reprogramar y cancelar. Por eso el cliente se modela en **dos tablas**: la ficha (Clients) existe siempre; la cuenta existe solo si la persona la crea.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador de la cuenta. Lo referencian AccountTokens.
`BusinessId`	`INT FK → Businesses`	La cuenta es por negocio: el acceso al panel del cliente se resuelve contra el slug del local.
`ClientId`	`INT NULL FK → Clients`	Vínculo con la ficha. **Nulo hasta verificar el correo**: una cuenta recién registrada no tiene ficha visible todavía. Es el único punto nulo intencional del modelo de acceso.
`Email`	`NVARCHAR(254)`	Correo de login. Único por negocio: es la credencial y el destino de verificación y reseteo.
`PasswordHash`	`NVARCHAR(400)`	Hash de la contraseña del cliente. Mismo criterio que OwnerAccounts: nunca el texto plano.
`EmailVerifiedAt`	`DATETIME2(0) NULL`	Cuándo se confirmó el correo. Nulo = cuenta pendiente. Es el dato que decide si el cliente puede operar: mientras sea nulo la cuenta existe pero no tiene ficha. `RF-25, D-18`
`CreatedAt`	`DATETIME2(0)`	Alta de la cuenta. Permite depurar registros nunca verificados.

COLUMNAS AGREGADAS POR LA MIGRACIÓN 002

Propiedad	Tipo	Uso
`PendingFirstName`	`NVARCHAR(60) NULL`	Nombre declarado por el cliente al autorregistrarse, guardado mientras la cuenta está pendiente. **Por qué:** sin `ClientId` no hay ficha donde guardarlo, y perderlo obligaría a pedirlo de nuevo al verificar. Se limpia cuando la verificación consume el enlace y crea la ficha.
`PendingLastName`	`NVARCHAR(60) NULL`	Apellido, con el mismo ciclo de vida. Ambas quedan nulas en toda cuenta que no provenga de autorregistro.

Índice único filtrado `UX_ClientAccounts_Client`: una ficha se vincula a lo sumo a una cuenta. Expresa `RF-26` : **no hay fusiones automáticas**. Si dos cuentas resultan ser la misma persona, la unificación es una decisión humana, no un efecto colateral del alta.

AccountTokens — tokens de un solo uso de la cuenta

Por qué existe: verificar un correo y restablecer una contraseña son operaciones **sin sesión previa**: necesitan un secreto temporal que viaje por correo. Esta tabla lo guarda, y su diseño garantiza dos cosas: el token es de un solo uso y en la base **nunca hay un valor que sirva para entrar**.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador del token.
`ClientAccountId`	`INT FK → ClientAccounts`	Cuenta destinataria. El token solo sirve para esa cuenta.
`Purpose`	`TINYINT — AccountTokenPurpose`	1 = verificación de correo, 2 = reseteo de contraseña. **Por qué:** separa propósitos para que un enlace de verificación no pueda usarse como reseteo. Un token es válido solo para lo que se emitió.
`TokenHash`	`BINARY(32) UNIQUE`	SHA-256 del token, nunca el token. **Por qué 32 bytes fijos:** es exactamente el tamaño del digest, así que la columna no admite otro formato y la comparación es directa. Único, para que un hash no pueda validar dos registros. Si alguien lee la base, no obtiene enlaces utilizables.
`ExpiresAt`	`DATETIME2(0)`	Vencimiento del enlace. Acota la ventana de un token filtrado por correo. Obligatorio: un token sin vencimiento es un acceso permanente.
`UsedAt`	`DATETIME2(0) NULL`	Cuándo se consumió. Nulo = sin usar. Es el campo que hace el token **de un solo uso**: consumido, no vuelve a aceptarse aunque alguien conserve el enlace.
`CreatedAt`	`DATETIME2(0)`	Emisión. Permite ordenar tokens y auditar reemisiones.

AccountInvitations — la invitación a crear cuenta

Por qué existe: es el camino inverso al autorregistro: la dueña invita a un cliente que **ya tiene ficha** a crear su cuenta. Se separa de AccountTokens porque el sujeto es distinto —la invitación nace de un `ClientId` existente, no de una cuenta— y porque tiene un estado que los tokens no necesitan: **revocar**.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador de la invitación.
`BusinessId`	`INT FK → Businesses`	Negocio que invita. Da contexto al enlace y delimita el alcance.
`ClientId`	`INT FK → Clients`	Ficha invitada. Debe existir: no se invita a quien no está en la libreta.
`Email`	`NVARCHAR(254)`	Correo al que se envió la invitación, copiado de la ficha al emitir. Se conserva aparte para poder auditar a dónde fue, aunque la ficha cambie de correo después.
`TokenHash`	`BINARY(32) UNIQUE`	Mismo criterio que AccountTokens: hash de 32 bytes, único, nunca el valor utilizable.
`ExpiresAt`	`DATETIME2(0)`	Vencimiento de la invitación.
`UsedAt`	`DATETIME2(0) NULL`	Consumo. Junto con `RevokedAt` define los tres estados posibles: vigente, usada, revocada.
`RevokedAt`	`DATETIME2(0) NULL`	Anulación por la dueña (se equivocó de cliente, el cliente pidió no recibirla). **Por qué hace falta además de `UsedAt`:** una invitación revocada y una usada son hechos distintos y deben poder distinguirse en la auditoría.
`CreatedAt`	`DATETIME2(0)`	Emisión de la invitación.

AppointmentLinks — el enlace privado del turno

Por qué existe: el cliente que reservó sin cuenta recibe un enlace privado para confirmar o cancelar su turno, y para avisar una falta `RF-12, RF-24`. Ese enlace es su credencial: no hay usuario ni contraseña, hay un token largo y difícil de adivinar. Es la pieza que hace viable "reservar sin cuenta" sin dejar los turnos a la vista de cualquiera.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador del enlace.
`BookingId`	`INT FK → Bookings`	Anotación que el enlace gobierna. Apunta a la anotación y no al turno porque la acción del cliente (confirmar, cancelar, avisar falta) es siempre **sobre su lugar**, no sobre la clase entera `Regla 3`.
`TokenHash`	`BINARY(32) UNIQUE`	Hash del token que viaja en la URL. Mismo criterio de seguridad que los otros tokens.
`ProtectedToken`	`NVARCHAR(600)`	Token **protegido pero recuperable** con la clave del servidor. **Por qué existe habiendo hash:** la dueña tiene que poder reenviar el enlace de un cliente que lo perdió, y un hash no permite reconstruir la URL. Es el único token del esquema que se guarda de forma reversible, y por eso se protege en lugar de hashearse.
`ExpiresAt`	`DATETIME2(0)`	Vencimiento del enlace. Acota la vigencia: un enlace de un turno viejo no debe seguir operando.
`RevokedAt`	`DATETIME2(0) NULL`	Anulación explícita (por ejemplo, tras un reenvío o una cancelación). Corta el acceso sin borrar el registro.
`CreatedAt`	`DATETIME2(0)`	Emisión. Permite saber cuándo se generó el enlace vigente.

`IX_AppointmentLinks_Booking` resuelve la consulta inversa —"los enlaces de esta anotación"— necesaria para reenviar o revocar.

6. Anotaciones y notas

Bookings — la anotación de un cliente en un turno

Por qué existe: expresa la `Regla 3`, la más importante del modelo: **el estado corresponde al cliente, no al turno**. En un turno con cupo mayor a 1, cada alumno tiene su propia confirmación, asistencia o ausencia. Aquí también vive el **precio acordado** `D-20`, y la distinción entre *fija* y *suelta*: una anotación fija nace de una inscripción; una suelta entra a una clase puntual. Esta tabla es, literalmente, la línea de la libreta.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador de la anotación. Lo referencian AppointmentLinks y Payments.
`BusinessId`	`INT FK → Businesses`	Aísla las anotaciones por negocio.
`AppointmentId`	`INT FK → Appointments`	Turno en el que se anota la persona `Regla 4`.
`ClientId`	`INT FK → Clients`	Quién. Junto con el turno define la línea de agenda.
`EnrollmentId`	`INT NULL FK → Enrollments`	Inscripción que ampara la anotación, cuando es fija. Nulo en las sueltas y en los turnos individuales. Es lo que permite saber si el alumno tenía lugar reservado esa semana `D-16`.
`Modality`	`TINYINT NULL — BookingModality`	1 = *Fija*, 2 = *Suelta*. Nulo en turnos individuales, donde la distinción no existe. Es una columna aparte y no algo derivado de `EnrollmentId` porque una suelta podría a futuro tener origen distinto; hoy `CK_Bookings_FixedHasEnrollment` exige que una fija tenga inscripción, impidiendo una "fija sin abono" imposible.
`Status`	`TINYINT, 1 — BookingStatus`	1 = *Pendiente*, 2 = *Confirmado*, 3 = *Atendido*, 4 = *Ausente*, 5 = *Cancelado*. Es el estado del cliente en ese turno. Nótese que ***Cobrado* no es un estado**: se deriva de un pago no anulado, porque asistencia y cobro son independientes `D-20`.
`AbsenceSource`	`TINYINT NULL — AbsenceSource`	1 = la dueña, 2 = el cliente por el enlace privado `RF-12, RF-24`. Nulo si no hubo ausencia. **Por qué:** la `Regla 18` hace que avisar libere el lugar esa semana; registrar la fuente permite distinguir el aviso del cliente de la falta que registra el local.
`CancelledBy`	`TINYINT NULL — CancelledBy`	1 = el cliente, 2 = la dueña `RF-22`. Nulo si no hubo cancelación. Permite reportes de cancelación por origen y protege de discusiones sobre quién canceló.

Propiedad	Tipo	Uso
AgreedPrice	DECIMAL(12,2) NULL	**Precio acordado**: se copia del tarifario al anotar y los cambios posteriores del tarifario **nunca** lo alteran. Es la razón por la que el precio vive en la anotación y no se lee del servicio al cobrar. Nulable porque la anotación puede existir antes de acordar el importe (una clase suelta sin cobrar todavía).
PriceReason	NVARCHAR(200) NULL	Motivo cuando el precio acordado difiere del tarifario (descuento, arreglo especial). Es el campo que hace auditable el desvío sin prohibirlo: el sistema valida reservas pero no impide registros `D-10`.
CreatedAt	DATETIME2(0)	Cuándo se anotó.
ConfirmedAt	DATETIME2(0) NULL	Cuándo el cliente confirmó.
CancelledAt	DATETIME2(0) NULL	Cuándo se canceló. `CK_Bookings_Cancelled` lo obliga a estar presente **si y solo si** el estado es Cancelado: evita la contradicción de una anotación cancelada sin fecha o una activa con fecha de cancelación.
AttendanceRecordedAt	DATETIME2(0) NULL	Cuándo se marcó la asistencia o la ausencia. Separa el momento del hecho del momento de carga, que es justamente lo que la libreta no permite.

ÍNDICES QUE CODIFICAN REGLAS

- `IX_Bookings_Appointment` y `IX_Bookings_Client`: los dos recorridos habituales —la lista de un turno y la historia de un cliente—.
- `UX_Bookings_FixedPerSession` (único filtrado sobre `AppointmentId, EnrollmentId` donde hay inscripción): regenerar la semana **no duplica** una reserva fija `RF-06`. Sin esta restricción, cada regeneración sumaría un lugar fantasma al alumno.
- `UX_Bookings_ActiveClient` (único filtrado donde `Status <> 5`): un cliente tiene a lo sumo **una anotación activa** por turno. Cancelar libera el lugar `Regla 5`, y por eso el filtro excluye las canceladas: se puede volver a anotar al mismo cliente después de cancelar, pero no dos veces a la vez.

ClientNotes — la observación para la próxima atención

Por qué existe: `RF-11` pide registrar una observación para la próxima atención, visible solo para la dueña. Es el reemplazo del papelito pegado en la hoja: "la próxima traer el secador", "avisar del cambio de precio".

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador de la nota.
`ClientId`	`INT FK → Clients`	Cliente al que refiere la observación. Es la dimensión de `IX_ClientNotes_Client`: las notas se consultan al abrir la ficha o al atender.
`AppointmentId`	`INT NULL FK → Appointments`	Turno durante el cual se tomó la nota. **Nulable a propósito:** hay observaciones que surgen fuera de un turno (una llamada, un mensaje), y obligarlas a un turno inventaría un hecho que no ocurrió.
`Text`	`NVARCHAR(1000)`	Contenido de la observación. 1000 caracteres es una nota de trabajo extensa, no un campo de texto libre ilimitado.
`CreatedAt`	`DATETIME2(0)`	Cuándo se escribió. Ordena las notas y permite mostrarlas de la más reciente hacia atrás.

Nota sobre el modelo: esta tabla **no lleva `BusinessId`**, a diferencia del resto de las tablas de inquilino. No es un olvido: el negocio ya queda determinado por `ClientId`, y una nota no se consulta suelta sino siempre a través de su cliente o su turno. Se documenta para que nadie lo "corrija" agregando una columna redundante.

7. Inscripciones y abono

Enrollments — la inscripción a clases fijas

Por qué existe: el abono mensual reserva un lugar en las clases fijas `D-13, Regla 16`. La inscripción es la **relación** que lo representa, y su ciclo de vida es explícito porque pasa por estados que el relevamiento distingue con cuidado: nace *pendiente de pago* reteniendo sus lugares (D-21), se *activa* con el primer pago, y termina por *anulación* de la dueña o por *baja* del alumno `RF-05, RF-20, RF-29`. La cobertura mensual concreta no vive aquí, sino en MembershipPeriods.

Propiedad	Tipo	Uso
Id	INT IDENTITY PK	Identificador de la inscripción. Lo referencian EnrollmentFixedClasses, Bookings, Payments y MembershipPeriods.
BusinessId	INT FK → Businesses	Aísla las inscripciones por negocio.
ClientId	INT FK → Clients	Alumno inscripto. Debe existir ficha: la inscripción no crea clientes.
WeeklyFrequency	INT 1..7	Cuántas clases fijas por semana tiene derecho. Es la dimensión con la que se buscó el precio en MembershipPrices y, en la práctica, cuántos lugares retiene la inscripción.
Status	TINYINT, 1 — EnrollmentStatus	1 = *Pendiente de pago* (retiene lugares, D-21), 2 = *Activa* (al menos un período pagado), 3 = *Anulada* (la dueña la anula, libera lugares), 4 = *Baja* (la dueña termina la membresía, libera lugares, conserva historia). **Por qué cuatro y no un bit activo/inactivo:** "anulada" y "de baja" tienen efectos iguales pero significados distintos, y el reporte de resultados y la auditoría necesitan distinguirlos.
AgreedPrice	DECIMAL(12,2)	Precio mensual acordado, copiado del tarifario al inscribir. Igual que en Bookings, los cambios de tarifa posteriores no lo alteran. `CHECK (> 0)`.
PriceReason	NVARCHAR(200) NULL	Motivo del precio diferencial (promoción, arreglo). Hace auditable el desvío.
ReferenceDay	TINYINT NULL 1..31	Día de referencia para calcular vencimientos `D-23`. Se fija al activar la membresía. **Por qué un día y no una fecha:** el abono es mensual y se renueva en el mismo número de día; guardar el día permite derivar cada período y evita recalcularlo de los pagos. Es 1..31 y no 1..28 porque el modelo acepta el día declarado y la lógica de calendario resuelve los meses cortos.
CreatedAt	DATETIME2(0)	Alta de la inscripción. Con estado Pendiente es el punto de partida de la retención de lugares.
ActivatedAt	DATETIME2(0) NULL	Cuándo se activó (primer pago). Nulo mientras esté pendiente.
AnnulledAt	DATETIME2(0) NULL	Cuándo la dueña anuló. Nulo salvo estado 3.

Propiedad	Tipo	Uso
`TerminatedAt`	`DATETIME2(0) NULL`	Cuándo se dio de baja. Nulo salvo estado 4. Las tres fechas existen para que cada transición quede fechada y el reporte histórico no dependa del estado actual.

EnrollmentFixedClasses — los lugares fijos de la inscripción

Por qué existe: "el abono reserva el lugar del alumno en sus clases fijas todas las semanas" **`Regla 16`**. Esta tabla dice **en qué clases**, y lo hace **con vigencia** en lugar de sobrescribir: el alumno cambió de clase en marzo, y la historia de las semanas anteriores tiene que seguir explicando por qué estaba anotado donde estaba.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador del lugar fijo.
`EnrollmentId`	`INT FK → Enrollments`	Inscripción que retiene el lugar.
`GridClassId`	`INT FK → GridClasses`	Clase de la grilla asignada. Es la dimensión de `IX_EnrollmentFixedClasses_Grid`, porque la consulta caliente es la inversa: "qué alumnos tienen lugar fijo en esta clase de esta semana" — el cálculo de cupo de una sesión **`Regla 2`**.
`ValidFrom / ValidTo`	`DATE / DATE NULL`	Vigencia del lugar. `ValidTo` nulo significa vigente. `CHECK (ValidTo IS NULL OR ValidTo >= ValidFrom)` impide vigencias invertidas. Es el mecanismo que permite cambiar de clase sin reescribir el pasado y que el índice incluya `ValidFrom/ValidTo` para resolver "quién tiene lugar esa semana" sin tocar la tabla.

MembershipPeriods — cada mes cubierto por un pago

Por qué existe: el estado del abono en una fecha (*Vigente* / *Vencido*) es una **derivación**, no un dato mutable. Guardar la cobertura como períodos fechados evita el problema clásico del campo "vencimiento" que hay que actualizar a mano y que queda desactualizado. Cada fila es un mes de abono, y nace de un pago concreto `Regla 16, D-13`.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador del período.
`EnrollmentId`	`INT FK → Enrollments`	Inscripción cubierta. `IX_MembershipPeriods_Enrollment` ordena por `ExpiresOn`, que es exactamente la consulta "¿está vigente hoy?" y "¿cuándo vence?".
`CoverageStart`	`DATE`	Primer día cubierto. Es `DATE` y no `DATETIME2` porque la vigencia de un abono es una cuestión de días, no de horas.
`ExpiresOn`	`DATE`	Último día cubierto. `CHECK (ExpiresOn > CoverageStart)`: un período de duración nula o negativa no cubre nada y sería un error de datos, no un caso válido. Si un período se anula (por ejemplo, por un reembolso), se elimina o rehace el período: la vigencia es un hecho y no lleva marca de baja.
`PaymentId`	`INT FK → Payments, UNIQUE`	Pago que originó la cobertura. **Único:** un pago no puede cubrir dos veces el mismo mes. Es el vínculo que hace reconstruible el abono desde la caja.
`CreatedAt`	`DATETIME2(0)`	Cuándo se registró la cobertura.

8. Caja: cobros, anulaciones, devoluciones y gastos

Payments — todo lo que entra

Por qué existe: es la caja. Un solo lugar para todos los conceptos (turno individual, atención sin turno, clase suelta, abono) porque el reporte de resultados necesita ver todo el dinero junto `RF-13`. La decisión **D-20** se materializa en `CK_Payments_Reference`: **cada pago identifica su origen concreto** —un abono apunta a la inscripción, todo lo demás a una anotación—. Sin eso, el importe no sería explicable y el reporte no podría cruzar cobros contra atenciones.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador del pago. Lo referencian PaymentAnnulments, Refunds y MembershipPeriods.
`BusinessId`	`INT FK → Businesses`	Aísla la caja por negocio; es la columna de `IX_Payments_BusinessPaidAt` para los reportes por período.
`ClientId`	`INT NULL FK → Clients`	Quién pagó. **Nulable** porque hay ingresos sin cliente identificado (una venta de mostrador, un ajuste de caja). La columna permite atribuir el cobro cuando se conoce.
`Concept`	`TINYINT — PaymentConcept`	1 = turno individual, 2 = atención sin turno, 3 = clase suelta, 4 = abono. Es el campo que decide qué referencia aplica y cómo se clasifica el ingreso en los reportes.
`BookingId`	`INT NULL FK → Bookings`	Anotación cobrada. Presente en conceptos 1, 2 y 3. `IX_Payments_Booking` es **filtrado** (`WHERE BookingId IS NOT NULL`) porque solo esos conceptos tienen anotación, y el índice es más chico y preciso.
`EnrollmentId`	`INT NULL FK → Enrollments`	Inscripción cobrada. Presente solo en concepto 4. La restricción `CK_Payments_Reference` garantiza que **nunca** estén las dos referencias a la vez ni ninguna: un cobro de abono sin inscripción no tiene a quién cubrir, y un cobro de turno sin anotación no tiene qué explicar `Regla 12`.
`Amount`	`DECIMAL(12,2)`	Importe efectivamente cobrado. `CHECK (> 0)`: el esquema no acepta importes negativos ni cero. Un error de carga se corrige con una **anulación** o una **devolución**, no con un pago en negativo, y por eso existen las dos tablas siguientes.
`ListPrice`	`DECIMAL(12,2) NULL`	Precio de lista vigente al momento del cobro. Guardarlo junto al importe permite reportar descuentos sin recalcular tarifas históricas ni asumir que el precio de hoy es el de ayer.
`PriceReason`	`NVARCHAR(200) NULL`	Motivo del desvío respecto de la lista. Misma idea que en Bookings y Enrollments: se registra el desvío, no se bloquea.
`PaidAt`	`DATETIME2(0)`	**Cuándo se pagó** (fecha del hecho). Es la fecha con la que se imputa el ingreso al período del reporte.

Propiedad	Tipo	Uso
`RecordedAt`	`DATETIME2(0)`	**Cuándo se cargó** en el sistema. Separar ambas es lo que permite detectar cargas tardías y reconstruir el orden real de los hechos, algo imposible en la libreta.

PaymentAnnulments — la anulación de un cobro mal registrado

Por qué existe: `Payments.Amount > 0` prohíbe el pago negativo, así que corregir un importe mal cargado exige **anular** y volver a registrar. La anulación conserva el pago original: la caja no borra historia, la explica.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador de la anulación.
`PaymentId`	`INT FK → Payments, UNIQUE`	Pago anulado. **Único:** un pago se anula una sola vez. La anulación es un hecho definitivo, no un estado que se alterna; si es necesario revertirla, corresponde un pago nuevo.
`Reason`	`NVARCHAR(300)`	Motivo. Obligatorio: una anulación sin motivo deja un hueco inexplicable en la caja.
`AnnulledAt`	`DATETIME2(0)`	Cuándo se anuló. Fecha el hecho para la auditoría del período.
`AnnulledBy`	`NVARCHAR(60)`	Quién anuló, por nombre de usuario. Hoy hay una sola cuenta por negocio, pero el dato se guarda igual: la restricción puede cambiar y la trazabilidad no debe depender de ella.

Refunds — la devolución de dinero

Por qué existe: es un hecho **distinto** de la anulación. Anular es decir "ese cobro no debió registrarse"; devolver es decir "ese cobro fue real y ahora salió plata". Confundirlos falsearía el reporte: la anulación saca el ingreso del período, la devolución lo deja y agrega un egreso de dinero real.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador de la devolución.
`PaymentId`	`INT FK → Payments`	Pago devuelto. **Sin restricción de unicidad**, a diferencia de la anulación: una devolución puede ser parcial y puede haber más de una sobre el mismo pago hasta cubrir el importe.
`Amount`	`DECIMAL(12,2)`	Importe devuelto. Separado del importe del pago precisamente para permitir devoluciones parciales. `CHECK (> 0)`.
`RefundedOn`	`DATE`	Fecha efectiva de la devolución. Es `DATE` porque se imputa por día de caja.
`Reason`	`NVARCHAR(300)`	Motivo. Obligatorio, igual que en la anulación: un egreso sin explicación no es auditable.
`RecordedAt`	`DATETIME2(0)`	Cuándo se cargó la devolución, separado de cuándo ocurrió.

Expenses — lo que sale

Por qué existe: el resultado del período es ingresos menos gastos `RF-16`. Esta tabla es la mitad del cálculo, junto con Payments.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador del gasto.
`BusinessId`	`INT FK → Businesses`	Aísla los gastos por negocio; con `SpentOn` forma `IX_Expenses_BusinessDay` para los reportes por período.
`ExpenseTypeId`	`INT FK → ExpenseTypes`	Clasificación del gasto. Obligatorio: un gasto sin tipo no se puede agrupar en el reporte.
`SpentOn`	`DATE`	Fecha del gasto. Se imputa por día, consistente con el resto de las imputaciones de caja.
`Amount`	`DECIMAL(12,2)`	Importe. `CHECK (> 0)` por el mismo criterio que en Payments: el signo lo da la tabla, no el número.
`Description`	`NVARCHAR(300)`	Detalle del gasto. Obligatorio: es lo único que distingue dos gastos del mismo tipo y fecha.
`CreatedAt`	`DATETIME2(0)`	Cuándo se cargó, separado de cuándo ocurrió.

9. Insumos y avisos

Supplies — el depósito declarado

Por qué existe: la `Regla 8` y `D-03` son tajantes: el sistema registra **existencias declaradas, no calcula consumo**. El consumo real no se registra y varía, y las cosas se reponen al verlas gastadas. Por eso esta tabla guarda un número declarado y una fecha de declaración, y el sistema se limita a **avisar** cuándo hay que volver a mirar `RF-18`.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador del insumo.
`BusinessId`	`INT FK → Businesses`	Aísla el depósito por negocio.
`Name`	`NVARCHAR(100)`	Nombre del insumo ("coloración 7.3", "mat de yoga"). Único por negocio: dos insumos con el mismo nombre impedirían saber cuál se está declarando.
`CurrentStock`	`DECIMAL(10,2)`	Existencia declarada actual. Es `DECIMAL` y no entero porque hay insumos que se cuentan por fracción (litros, gramos). El sistema **nunca** lo modifica por su cuenta: cambia solo cuando una persona declara un conteo. `CHECK (>= 0)`: una existencia negativa es imposible.
`MinimumStock`	`DECIMAL(10,2)`	Umbral a partir del cual el insumo está bajo. Es lo que permite que el aviso diga **qué** revisar y no solo "revisá el depósito". `CHECK (>= 0)`.
`LastDeclaredAt`	`DATETIME2(0)`	Cuándo se declaró el conteo. Es el dato con el que el aviso periódico (`StockReviewEveryDays`) decide si ya toca insistir; sin esta fecha, el aviso no tendría desde cuándo contar.
`IsActive`	`BIT NOT NULL, 1`	Baja lógica. Un insumo que ya no se usa deja de aparecer y de generar avisos, sin perder su historia.

Notifications — la bandeja de salida de correo (outbox)

Por qué existe: el sistema genera trece clases de aviso —recordatorios, confirmaciones, suspensión de clase, vencimiento de abono, resumen diario, revisión de existencias, verificación, reseteo e invitación— `RF-17, RF-18, RF-19`. Enviar correo dentro de la transacción de negocio es frágil: si el proveedor falla, la operación de la dueña se cae con él. Por eso el envío se modela como **outbox**: la notificación se escribe como dato y un despachador la envía después. La columna `DedupKey` es lo que hace que reintentar sea seguro.

Propiedad	Tipo	Uso
`Id`	`INT IDENTITY PK`	Identificador de la notificación. Es también el desempate final del orden de despacho (`IX_Notifications_Due`).
`BusinessId`	`INT NULL FK → Businesses`	Negocio al que pertenece el aviso. **Nulable** porque la verificación de correo y el reseteo de contraseña ocurren antes de resolver por completo el contexto del negocio, y esos avisos no deben quedar huérfanos por eso.
`Type`	`TINYINT — NotificationType`	Qué aviso es (1..13). Determina la plantilla y participa de la clave de deduplicación: dos avisos distintos sobre el mismo hecho **no** son duplicados.
`Recipient`	`NVARCHAR(254)`	Destinatario del correo. Se copia al emitir, no se resuelve al enviar: el aviso debe salir a donde se decidió que saliera, aunque la ficha cambie después.
`Subject`	`NVARCHAR(200)`	Asunto del correo, guardado con el aviso. **Por qué se persiste y no se regenera:** el aviso es un hecho fechado; si la plantilla cambia, un pendiente viejo debe enviarse tal como se emitió.
`BodyHtml`	`NVARCHAR(MAX)`	Cuerpo del correo ya renderizado, por la misma razón que el asunto. `MAX` porque el resumen diario y los avisos con detalle son documentos largos.
`DedupKey`	`NVARCHAR(150) UNIQUE`	Clave de deduplicación: **hace idempotente cada aviso a través de reintentos**. Combina tipo y hecho (por ejemplo, recordatorio del turno 42). Es único, así que reintentar el despacho no puede producir un segundo correo al cliente. Es la razón por la que el dispatcher puede ser agresivo sin molestar.
`Status`	`TINYINT, 1 — NotificationStatus`	1 = *Pending*, 2 = *Sent*, 3 = *Failed* (falla definitiva o presupuesto de reintentos agotado; terminal, nunca se reclama), 4 = *Dispatching* (reclamado y enviándose ahora; nunca se recupera solo, lo resuelve una persona), 5 = *ReviewRequired* (resultado del envío desconocido; decisión manual). Los dos últimos estados existen porque **no se puede reintentar a ciegas** un correo que quizá salió: en el peor caso el cliente recibe dos recordatorios, y por eso la incertidumbre se escala a una persona.
`Attempts`	`INT NOT NULL, 0`	Cantidad de intentos. Es el contador que agota el presupuesto de reintentos y mueve el aviso a *Failed*.

Propiedad	Tipo	Uso
`LastError`	`NVARCHAR(500) NULL`	Último error del proveedor. Es lo que hace diagnosticable una fila en *Failed* sin buscar en logs.
`CreatedAt`	`DATETIME2(0)`	Emisión. Con `NextAttemptAt` nulo, es el instante efectivo de vencimiento de la fila.
`SentAt`	`DATETIME2(0) NULL`	Cuándo se envió. Nulo mientras no esté *Sent*.

COLUMNAS AGREGADAS POR LA MIGRACIÓN 003

Por qué son tres columnas nulables y no un rediseño: los siete productores de avisos previos a T23 no conocen estas columnas. Siendo nulables, siguen insertando sin tocarlas, `SQL` las deja nulas, y el instante efectivo de esa fila es simplemente `CreatedAt`. La migración es aditiva y no rompe a nadie.

Propiedad	Tipo	Uso
`NextAttemptAt`	`DATETIME2(0) NULL`	Instante más temprano en que puede intentarse un reintento. Nulo = el vencimiento efectivo es `CreatedAt`. Es lo que implementa la **espera entre reintentos** sin un proceso aparte.
`ClaimedAt`	`DATETIME2(0) NULL`	Cuándo se tomó el reclamo actual. No nulo solo en los estados *Dispatching* y *ReviewRequired*. Junto con la clave de despacho, es la trazabilidad que una persona necesita para resolver a mano un aviso en estado incierto.
`DispatchKey`	`UNIQUEIDENTIFIER NULL`	Token de reclamo del despacho en curso. **Toda finalización debe coincidir con él**: si un envío lento termina después de que su reclamo fue tomado por otro, la finalización obsoleta no escribe nada en lugar de pisar al dueño actual. Es el mecanismo que evita la doble escritura sin bloqueos largos.

`IX_Notifications_Due` sobre `(Status, NextAttemptAt, CreatedAt, Id)`: el barrido de vencimientos filtra por estado y ordena por `COALESCE(NextAttemptAt, CreatedAt), Id`; el índice mantiene contiguas las filas de un estado y lleva las dos claves de orden más el desempate. Es **deliberadamente no filtrado**: un filtro sobre `Status = 1` se mantendría en cada cambio de estado, y el índice sin filtro también sirve las consultas por estado del visor de outbox, sin costo extra de escritura.

10. Síntesis

Leído en conjunto, el esquema sostiene seis decisiones que explican casi todas sus propiedades:

Decisión	Dónde vive en el esquema
El cliente es un caso particular del servicio con cupo 1	Un solo modelo de turno (`Appointments.Kind` + `MaxCapacity`) cubre turno individual, atención sin turno y clase grupal. No hay tablas paralelas.
El estado es del cliente, no del turno	`Appointments.Status` es global; `Bookings.Status` es individual. Cada alumno tiene su confirmación, asistencia o ausencia.
Los precios se copian, no se referencian	`Bookings.AgreedPrice`, `Enrollments.AgreedPrice`, `Payments.ListPrice`. Cambiar el tarifario no reescribe el pasado.
La incertidumbre se escala a una persona	`NotificationStatus.Dispatching` y `.ReviewRequired`: nada se reintenta a ciegas.
Las reglas se imponen en la base, no solo en la aplicación	`CHECK`s de forma y referencia, índices únicos filtrados para idempotencia y unicidad de anotación activa.
Nada se borra: se desactiva o se anula	`IsActive` en catálogos, `PaymentAnnulments`, `AnnulledAt`/`TerminatedAt`, vigencias con `ValidFrom`/`ValidTo`.

INVENTARIO DE TABLAS

#	Tabla	Una frase
1	`Businesses`	El negocio y sus parámetros de operación.
2	`OwnerAccounts`	La cuenta única de acceso al panel.
3	`SystemSettings`	Clave/valor de la instalación (reservada).
4	`ExpenseTypes`	Catálogo de tipos de gasto del negocio.
5	`Services`	Tarifario: precio, duración y cupo.
6	`MembershipPrices`	Precio del abono según frecuencia semanal.
7	`Workers`	Quién atiende.
8	`WorkerSchedules`	Horario de atención semanal por trabajador.
9	`WorkerBreaks`	Pausas no laborables semanales.
10	`GridClasses`	La grilla recurrente de clases.
11	`Appointments`	El turno concreto y su estado global.
12	`Clients`	La ficha del cliente.
13	`ClientAccounts`	La cuenta opcional de autoservicio.
14	`AccountTokens`	Tokens de un solo uso de la cuenta.
15	`AccountInvitations`	Invitación de la dueña a crear cuenta.
16	`Bookings`	La anotación del cliente en el turno.
17	`ClientNotes`	Observación para la próxima atención.

#	Tabla	Una frase
18	`AppointmentLinks`	Enlace privado del turno.
19	`Enrollments`	Inscripción a clases fijas.
20	`EnrollmentFixedClasses`	Los lugares fijos, con vigencia.
21	`Payments`	Todo lo que entra.
22	`PaymentAnnulments`	Anulación de un cobro mal registrado.
23	`Refunds`	Devolución de dinero.
24	`MembershipPeriods`	Cada mes de abono cubierto por un pago.
25	`Expenses`	Lo que sale.
26	`Supplies`	Depósito declarado.
27	`Notifications`	Bandeja de salida de correo.